

Week 1 Assignment – 5 Problems

Java Solutions Compilation

Problem 1: The Exam Hall Seat Duplication Checker

SeatDuplicationChecker.java

```
public class SeatDuplicationChecker {

    /**
     * Custom checked exception used when the seat data supplied
     * is missing or empty.
     * Concept: Checked Exceptions
     */
    static class InvalidSeatDataException extends Exception {
        public InvalidSeatDataException(String message) {
            super(message);
        }
    }

    public static void main(String[] args) {
        int[] seatBatchOne = {101, 102, 103, 102, 105};
        int[] seatBatchTwo = {101, 102, 103, 104, 105};
        int[] seatBatchEmpty = {};

        processSeatBatch(seatBatchOne, "Batch 1");
        processSeatBatch(seatBatchTwo, "Batch 2");
        processSeatBatch(seatBatchEmpty, "Batch 3 (Empty Input)");
    }

    /**
     * Handles a single batch of seat numbers: checks for duplicates
     * and handles both checked and unchecked exceptions.
     */
    private static void processSeatBatch(int[] seatNumbers, String batchLabel) {
        System.out.println("Checking " + batchLabel + ":");
        try {
            checkDuplicateSeats(seatNumbers);
        } catch (InvalidSeatDataException invalidSeatDataException) {
            System.out.println("Checked Exception Caught: " + invalidSeatDataException.getMessage());
        } catch (NullPointerException nullPointerException) {
            System.out.println("Unchecked Exception Caught: Seat array reference is null.");
        }
        System.out.println();
    }

    /**
     * Scans an array of seat numbers and prints any duplicate seat
     * numbers found by comparing every seat against every other seat.
     * Concept: Arrays, nested loops, conditional logic
     */
    private static void checkDuplicateSeats(int[] seatNumbers) throws InvalidSeatDataException {
        if (seatNumbers == null || seatNumbers.length == 0) {
            throw new InvalidSeatDataException("Seat number array is empty or missing.");
        }

        boolean duplicateFound = false;

        for (int i = 0; i < seatNumbers.length; i++) {
```

```
        for (int j = i + 1; j < seatNumbers.length; j++) {
            if (seatNumbers[i] == seatNumbers[j]) {
                System.out.println("Duplicate Seat Number Found: " + seatNumbers[i]);
                duplicateFound = true;
                break;
            }
        }
    }

    if (!duplicateFound) {
        System.out.println("No Duplicate Seats Found");
    }
}
```

Problem 2: The Typing Speed Test Accuracy Checker

TypingAccuracyChecker.java

```
public class TypingAccuracyChecker {

    /**
     * Custom checked exception used when the original passage and
     * the typed attempt are not the same length.
     * Concept: Checked Exceptions
     */
    static class UnequalLengthException extends Exception {
        public UnequalLengthException(String message) {
            super(message);
        }
    }

    public static void main(String[] args) {
        evaluateTypingAttempt("hello world", "hello worlt");
        evaluateTypingAttempt("coding", "coding");
        evaluateTypingAttempt("java", "jav"); // unequal length triggers checked exception
    }

    /**
     * Displays the attempt and handles the checked exception thrown
     * when the two strings are not the same length.
     */
    private static void evaluateTypingAttempt(String originalPassage, String typedPassage) {
        System.out.println("Original: \"" + originalPassage + "\"\ntTyped: \"" + typedPassage + "\"");
        try {
            checkTypingAccuracy(originalPassage, typedPassage);
        } catch (UnequalLengthException unequalLengthException) {
            System.out.println("Checked Exception Caught: " + unequalLengthException.getMessage());
        }
        System.out.println();
    }

    /**
     * Compares two equal-length strings character by character and
     * reports match count, accuracy percentage, and first mismatch.
     * Concept: String traversal, charAt(), percentage calculation
     */
    private static void checkTypingAccuracy(String original, String typed) throws UnequalLengthException {
        if (original.length() != typed.length()) {
            throw new UnequalLengthException("Original and typed text must be equal length. Original: "
                + original.length() + " chars, Typed: " + typed.length() + " chars.");
        }

        int totalCharacters = original.length();
        int matchedCharacters = 0;
        int firstMismatchPosition = -1;
        char originalMismatchChar = ' ';
        char typedMismatchChar = ' ';

        for (int index = 0; index < totalCharacters; index++) {
            char originalChar = original.charAt(index);
            char typedChar = typed.charAt(index);

            if (originalChar == typedChar) {
                matchedCharacters++;
            } else if (firstMismatchPosition == -1) {
                firstMismatchPosition = index + 1;
                originalMismatchChar = originalChar;
            }
        }
    }
}
```

```

        typedMismatchChar = typedChar;
    }
}

double accuracyPercentage = ((double) matchedCharacters / totalCharacters) * 100;

System.out.printf("Matched: %d/%d | Accuracy: %.2f%%", matchedCharacters, totalCharacters, accuracyPercentage);

if (firstMismatchPosition == -1) {
    System.out.println(" | No Mismatches");
} else {
    System.out.println(" | First Mismatch at position " + firstMismatchPosition +
        " ('" + originalMismatchChar + "' vs '" + typedMismatchChar + "')");
}
}
}

```

Problem 3: The Traffic Signal Streak Analyzer

TrafficSignalStreakAnalyzer.java

```
public class TrafficSignalStreakAnalyzer {

    /**
     * Custom checked exception used when the signal log contains a
     * character that is not a valid signal color.
     * Concept: Checked Exceptions
     */
    static class InvalidSignalException extends Exception {
        public InvalidSignalException(String message) {
            super(message);
        }
    }

    public static void main(String[] args) {
        analyzeSignalLog("RRGGGYRR");
        analyzeSignalLog("RRRRYYGG");
        analyzeSignalLog("RRGXGY"); // invalid character triggers checked exception
        analyzeSignalLog("");      // empty log triggers unchecked exception
    }

    /**
     * Displays the log being analyzed and handles both checked and
     * unchecked exceptions raised while finding the longest streak.
     */
    private static void analyzeSignalLog(String signalLog) {
        System.out.println("Analyzing signal log: \"" + signalLog + "\"");
        try {
            findLongestStreak(signalLog);
        } catch (InvalidSignalException invalidSignalException) {
            System.out.println("Checked Exception Caught: " + invalidSignalException.getMessage());
        } catch (IllegalArgumentException illegalArgumentException) {
            System.out.println("Unchecked Exception Caught: " + illegalArgumentException.getMessage());
        }
        System.out.println();
    }

    /**
     * Scans the signal log and reports the color and length of the
     * longest streak of consecutive identical characters.
     * Concept: String traversal, tracking a running maximum
     */
    private static void findLongestStreak(String signalLog) throws InvalidSignalException {
        if (signalLog == null || signalLog.length() == 0) {
            throw new IllegalArgumentException("Signal log cannot be empty.");
        }

        validateSignalCharacters(signalLog);

        char longestColor = signalLog.charAt(0);
        int longestLength = 1;

        char currentColor = signalLog.charAt(0);
        int currentLength = 1;

        for (int index = 1; index < signalLog.length(); index++) {
            if (signalLog.charAt(index) == currentColor) {
                currentLength++;
            } else {
                currentColor = signalLog.charAt(index);
            }
        }
    }

    private static void validateSignalCharacters(String signalLog) {
        for (int i = 0; i < signalLog.length(); i++) {
            char c = signalLog.charAt(i);
            if (c != 'R' & c != 'G' & c != 'Y') {
                throw new InvalidSignalException("Invalid signal color: " + c);
            }
        }
    }
}
```

```

        currentLength = 1;
    }

    if (currentLength > longestLength) {
        longestLength = currentLength;
        longestColor = currentColor;
    }
}

System.out.println("Longest Streak: '" + longestColor + "' repeated " + longestLength + " times");
}

/**
 * Validates that every character in the log is a recognized
 * signal color by comparing its ASCII value.
 * Concept: ASCII Character Codes, Checked Exceptions
 */
private static void validateSignalCharacters(String signalLog) throws InvalidSignalException {
    for (int index = 0; index < signalLog.length(); index++) {
        char signalCharacter = signalLog.charAt(index);
        int asciiValue = (int) signalCharacter;

        boolean isValidSignal = (asciiValue == 'R' || asciiValue == 'Y' || asciiValue == 'G');

        if (!isValidSignal) {
            throw new InvalidSignalException("Invalid signal character '" + signalCharacter +
                "' (ASCII " + asciiValue + ") found at position " + (index + 1));
        }
    }
}
}

```

Problem 4: The Warehouse Inventory Balancer

WarehouseInventoryBalancer.java

```
public class WarehouseInventoryBalancer {

    /**
     * Custom checked exception used when Section A and Section B
     * arrays do not have the same length.
     * Concept: Checked Exceptions
     */
    static class ArrayLengthMismatchException extends Exception {
        public ArrayLengthMismatchException(String message) {
            super(message);
        }
    }

    public static void main(String[] args) {
        analyzeInventorySafely(new int[]{20, 15, 30}, new int[]{25, 10, 30});
        analyzeInventorySafely(new int[]{20, 15}, new int[]{25, 10, 30}); // mismatch triggers checked exception
        analyzeInventorySafely(new int[]{}, new int[]{}); // empty arrays trigger unchecked exception
    }

    /**
     * Runs the inventory analysis and handles both checked and
     * unchecked exceptions that may occur.
     */
    private static void analyzeInventorySafely(int[] sectionA, int[] sectionB) {
        System.out.println("Analyzing Section A vs Section B");
        try {
            analyzeInventory(sectionA, sectionB);
        } catch (ArrayLengthMismatchException arrayLengthMismatchException) {
            System.out.println("Checked Exception Caught: " + arrayLengthMismatchException.getMessage());
        } catch (IllegalArgumentException illegalArgumentException) {
            System.out.println("Unchecked Exception Caught: " + illegalArgumentException.getMessage());
        }
        System.out.println();
    }

    /**
     * Compares total quantities of two sections and reports the
     * highest quantity item and its location.
     * Concept: Arrays, sum accumulation, tracking maximum with its index
     */
    private static void analyzeInventory(int[] sectionA, int[] sectionB) throws ArrayLengthMismatchException {
        if (sectionA.length == 0 || sectionB.length == 0) {
            throw new IllegalArgumentException("Section arrays must not be empty.");
        }

        if (sectionA.length != sectionB.length) {
            throw new ArrayLengthMismatchException("Section A and Section B must have equal length. " +
                "Section A length: " + sectionA.length + ", Section B length: " + sectionB.length);
        }

        int totalA = calculateSectionTotal(sectionA);
        int totalB = calculateSectionTotal(sectionB);

        String balanceStatus = (totalA == totalB) ? "Balanced" : "Not Balanced";

        int highestQuantity = sectionA[0];
        String highestSection = "Section A";
        int highestIndex = 0;
        for (int index = 0; index < sectionA.length; index++) {
```

```

        if (sectionA[index] > highestQuantity) {
            highestQuantity = sectionA[index];
            highestSection = "Section A";
            highestIndex = index;
        }
    }
    for (int index = 0; index < sectionB.length; index++) {
        if (sectionB[index] > highestQuantity) {
            highestQuantity = sectionB[index];
            highestSection = "Section B";
            highestIndex = index;
        }
    }

    System.out.println("Section A Total: " + totalA + " | Section B Total: " + totalB +
        " | Status: " + balanceStatus + " | Highest Quantity: " + highestQuantity +
        " (" + highestSection + ", Item " + (highestIndex + 1) + ")");
}

/**
 * Adds up all quantities in a section's array.
 */
private static int calculateSectionTotal(int[] sectionQuantities) {
    int total = 0;
    for (int index = 0; index < sectionQuantities.length; index++) {
        total += sectionQuantities[index];
    }
    return total;
}
}

```


Problem 5: The Movie Review Word Length Profiler

MovieReviewWordLengthProfiler.java

```
public class MovieReviewWordLengthProfiler {

    public static void main(String[] args) {
        classifyReviewSafely("This movie was absolutely fantastic and thrilling");
        classifyReviewSafely(" "); // blank review triggers unchecked exception
    }

    /**
     * Displays the review being profiled and handles the unchecked
     * exception thrown when the review text is empty.
     */
    private static void classifyReviewSafely(String review) {
        System.out.println("Review: \"" + review + "\"");
        try {
            classifyWordLengths(review);
        } catch (IllegalArgumentException illegalArgumentException) {
            System.out.println("Unchecked Exception Caught: " + illegalArgumentException.getMessage());
        }
        System.out.println();
    }

    /**
     * Splits a review into words and classifies each word as
     * Short (1-4 letters), Medium (5-8 letters), or Long (9+ letters).
     * Concept: String splitting (split()), counting/categorization
     */
    private static void classifyWordLengths(String review) {
        if (review == null || review.trim().isEmpty()) {
            throw new IllegalArgumentException("Review text cannot be empty.");
        }

        String[] words = review.trim().split("\\s+");

        int shortWordCount = 0;
        int mediumWordCount = 0;
        int longWordCount = 0;

        for (int index = 0; index < words.length; index++) {
            int wordLength = words[index].length();

            if (wordLength >= 1 && wordLength <= 4) {
                shortWordCount++;
            } else if (wordLength >= 5 && wordLength <= 8) {
                mediumWordCount++;
            } else {
                longWordCount++;
            }
        }

        System.out.println("Short: " + shortWordCount + " | Medium: " + mediumWordCount + " | Long: " + longWordCount);
    }
}
```